

Parcours : Ingénierie Informatique et Développement Logiciel

SUPPORT DU COURS

Théorie des Langages et Compilation

Année Universitaire : 2026 - 2027

Professeure ***Souad AMJAD***

Département d'Informatique

Théorie des Langages et COMPILATION

OBJECTIF

Ce document d'appui pédagogique est destiné aux étudiants de la Licence Fondamentale en Ingénierie Informatique et Développement Logiciel de l'Université Abdelmalek Essaâdi, Faculté des Sciences de Tétouan. Il constitue le support du cours Théorie des Langages et Compilation, dispensé au premier semestre.

Le cours de Théorie des Langages et Compilation porte sur les principaux concepts intervenant dans la définition, l'analyse et la traduction des langages de programmation. Son objectif est de fournir aux étudiants les connaissances fondamentales nécessaires à la compréhension des bases théoriques de l'informatique et des principaux mécanismes intervenant dans la conception des compilateurs.

À travers ce cours, les étudiants sont amenés à acquérir et à mettre en œuvre plusieurs notions et techniques fondamentales, notamment :

- Les principes de base de la compilation et des analyseurs lexicaux ;
- Les fondements de la programmation dirigée par la syntaxe ;
- La définition, l'utilisation et l'optimisation des expressions régulières ;
- Les techniques de reconnaissance et de contrôle de la validité syntaxique des programmes.

Le cours débute par une présentation générale du processus de compilation, de ses différentes phases et de leurs interactions.

Le deuxième chapitre est consacré à la théorie des langages formels. Il introduit les notions fondamentales d'alphabet, symbole, mot et langage, ainsi que les différentes opérations définies sur les mots et les langages.

Le troisième chapitre porte sur l'étude d'une classe importante de langages formels : les langages rationnels. Après avoir présenté leur définition et leurs

principales propriétés, différentes techniques permettant de les décrire sont étudiées, notamment les expressions régulières. Les théorèmes d'Arden et de l'étoile occupent une place importante dans ce chapitre, en raison de leur rôle dans la caractérisation et la représentation des langages rationnels.

Le quatrième chapitre est consacré à la notion de grammaire formelle. Une grammaire est définie par un ensemble de règles de production permettant de générer les phrases d'un langage et de déterminer si une phrase appartient à sa syntaxe. L'étude des grammaires permet ainsi d'aborder la structure syntaxique des langages et les mécanismes de génération et de reconnaissance des phrases.

Le cinquième chapitre présente les automates finis déterministes et non déterministes. Les définitions et propriétés nécessaires à leur compréhension et à leur utilisation sont introduites progressivement. Différentes transformations sont ensuite étudiées, notamment la déterminisation et la minimisation des automates. Le chapitre aborde également le calcul des langages reconnus par les automates à l'aide de la résolution de systèmes d'équations, notamment en utilisant le lemme d'Arden.

Le sixième chapitre est consacré aux principales phases de l'analyse d'un programme. Dans une première partie, l'analyse lexicale est étudiée. Elle consiste à examiner le texte source afin de le découper en unités lexicales appelées lexèmes, telles que les opérateurs, les mots réservés, les identificateurs, les constantes numériques et les autres éléments constitutifs du langage.

La seconde partie du chapitre est consacrée à l'analyse syntaxique, dont l'objectif est de vérifier que la séquence de lexèmes produite par l'analyse lexicale respecte les règles syntaxiques du langage. L'analyseur syntaxique reçoit ainsi en entrée le flux de lexèmes généré par l'analyse lexicale et produit une représentation structurée du programme, généralement sous la forme d'un arbre syntaxique ou d'un arbre de syntaxe abstraite.

Dans cette partie, les notions fondamentales nécessaires à la construction d'un analyseur syntaxique descendant sont étudiées, notamment le calcul des ensembles Premier (FIRST) et Suivant (FOLLOW), la construction de la table d'analyse ainsi que les grammaires LL(1).

De nombreux exemples et applications accompagnent les différentes définitions et notions théoriques afin de faciliter leur compréhension et leur mise en pratique. Chaque chapitre est également complété par une série d'exercices d'application et d'approfondissement, permettant aux étudiants de consolider progressivement leurs connaissances et de développer leur capacité à résoudre des problèmes liés à la théorie des langages et à la compilation.

Sommaire

Chapitre 1 ..... **8**
PRESENTATION GENERALE D'UN COMPILATEUR **8**
1. Introduction **8**
2. Définition **8**
3. Les principes fondamentaux de la compilation **8**
3. 1. Les propriétés de la compilation **8**
3. 2. La structure d'un compilateur **8**
3.2.1. Analyse lexicale 9
3.2.2. Analyse syntaxique 9
3.2.3. Analyse sémantique 9
3.2.4. La production du code objet 9
3.2.5. L'optimisation du code 9
3.2.6. Le traitement des erreurs 9
Chapitre 2 ..... **11**
GENERALITES SUR LES LANGAGES **11**
1. Introduction **11**
2. Mots **11**
3. Théorème de la Décomposition **12**
4. Relations entre mots **13**
5. Relations d'ordre **14**
5. 1. Ordre préfixiel **14**
5. 2. Ordre lexicographique **14**
5. 3. Ordre hiérarchique **14**
6. Langages et opérations **14**
6. 1. Opérateurs ensemblistes classiques **15**
6. 2. Opérateurs induits par la concaténation des mots **15**
Chapitre 3 ..... **16**
LES LANGAGES RATIONNELS **16**
1. Introduction **16**
2. Langages rationnels **16**
3. Théorème des parties finies **16**
4. Expressions rationnelles **17**
5. Définitions rationnelles **19**
6. Systèmes d'équations rationnelles **20**

6. 1. Lemme d'Arden **20**
6. 2. Algorithme de résolution **21**
7. Lemme de l'étoile (de la pompe) **21**
Chapitre 4 ..... **23**
LES GRAMMAIRES DE CHOMSKY **23**
1. Introduction **23**
2. Structures d'une grammaire **23**
3. Classification des grammaires **25**
3. 1. Type 0 : sans contraintes **25**
3. 2. Type 1 : contextuelle (sensible au contexte) **25**
3. 3. Type 2 : hors-contexte (algébrique) **25**
3. 4. Type 3 : linéaires (régulières à droite (resp. à gauche)) **26**
3. 5. Grammaire de Type 4 ..... **26**
4. Dérivations à gauche et à droite **26**
5. Cas des Langages Rationnels **27**
6. Représentation des grammaires type 2 avec la norme BNF ("Backus-Naur-Form") **27**
7. Arbres de dérivations **27**
7.1. Ambiguïté **28**
Chapitre 5 ..... **29**
LES AUTOMATES D'ÉTAT FINI **29**
1. Introduction **29**
2. Automates finis **29**
2.1 Fonctionnement d'un automate fini déterministe **29**
2.2. Représentation graphique **30**
2.3. Représentation matricielle **30**
3. Propriétés des états **31**
3. 1. État stérile **31**
3. 2. État accessible **31**
3. 3. État co-accessible **32**
3. 4. État utile **32**
3. 5. État inutile **32**
3. 6. État isolé **32**
4. Automate fini non déterministe **32**
5. Automate fini déterministe **32**
5.1 Déterminisation d'un automate sans ε-transitions **32**
5.1. Algorithme **33**

6. Minimisation d'un automate fini par raffinement successif..... 33
7. Automates finis et expressions régulières..... 35
Chapitre 6..... 37
ANALYSES LEXICALE, SYNTAXIQUE ET SEMANTIQUE..... 37
Partie I - Analyse lexicale 37
1. Introduction 37
2. Structure d'un analyseur lexical 38
2. Reconnaissance des lexèmes 38
4. Reconnaissance contextuelle..... 39
5. Génération d'analyseurs..... 39
6. Analyseur lexical codé directement 40
Partie 2 - Analyse syntaxique 40
1. Introduction 40
3. Calcul de PREMIER..... 42
2.1. Calcul de Premier pour un symbole 43
3. Calcul de SUIVANT..... 44
3. 1. Algorithme de construction des ensembles SUIVANT 44
4. Table d'analyse..... 45
4. 1. Construction d'analyseur 45
4. 2. Algorithme 45
5. Analyseur syntaxique dirigé par table..... 46
5. 1. Principe..... 46
5. 2. Algorithme d'analyse 47
6. Récursivité à gauche..... 49
6. 1. Elimination de la récursivité à gauche immédiate..... 49
7. Factorisation à gauche 50
7. 1. Algorithme 50

Chapitre 1

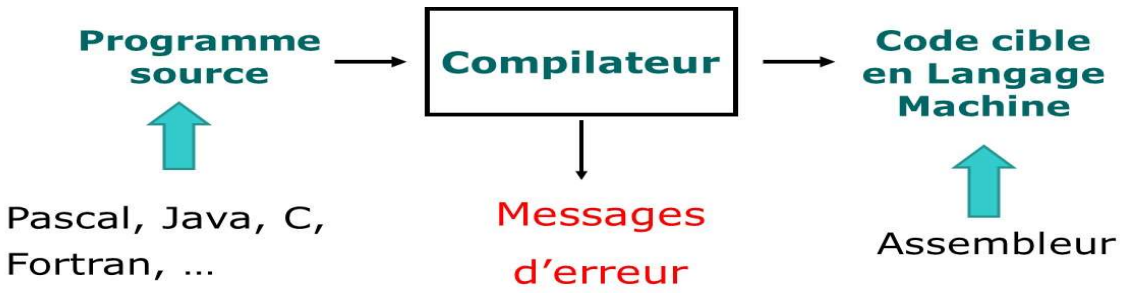
PRESENTATION GENERALE D'UN COMPILATEUR

1. Introduction

La compilation constitue la première étape de la chaîne de production de programmes.

2. Définition

La **compilation** permet la traduction d'un programme dit ***programme source*** écrit le plus souvent en langage de haut niveau vers un programme dit ***programme objet*** qui est soit directement le langage machine, soit le langage d'assemblage.



3. Les principes fondamentaux de la compilation

- Le compilateur doit conserver le sens du programme compilé.
- Le compilateur doit améliorer le code.

3. 1. Les propriétés de la compilation

- Le code produit doit être efficace (rapidité, mémoire).
- Le choix d'informations retournées en cas d'erreurs doit être judicieusement fait.
- La rapidité de la compilation doit être prise en compte.

3. 2. La structure d'un compilateur

Un compilateur peut être subdivisé en différentes phases :

3.2.1. Analyse lexicale

Reconnaissance des mots du langage ie. appréhension du vocabulaire : Elle lit la chaîne de caractères d'entrée, caractère par caractère, et la transforme en une suite de mots ou unités lexicales ;

3.2.2. Analyse syntaxique

Vérification de la syntaxe ie. appréhension de la grammaire : Elle vérifie si la suite d'unités lexicales reconnues par l'analyseur lexical est légale. L'analyse syntaxique dérive d'une structure hiérarchique d'unités lexicales qui peut être représentée par des arbres abstraits.

a. Analyse syntaxique ascendante : L'analyseur cherche, à partir de la phrase et à l'aide des règles de la grammaire, à remonter à l'élément initial (nous montons des feuilles vers la racine)

b. Analyse syntaxique descendante : L'analyseur cherche, à partir de l'élément initial et à l'aide des règles de la grammaire, à former la phrase donnée (nous descendons de la racine vers les feuilles)

3.2.3. Analyse sémantique

Vérification de la sémantique ie. appréhension du sens **:** Elle contrôle si le programme source a un sens. L'un des rôles importants de l'analyse sémantique est, en particulier, le contrôle de type : elle vérifie si les opérandes de chaque opérateur sont conformes aux spécifications du langage source (opérations arithmétiques entre nombres entiers, ou réels...).

3.2.4. La production du code objet

Le but final d'un compilateur est la production d'un programme objet.

3.2.5. L'optimisation du code

Le code produit par le compilateur doit être performant (ie. rapide à l'exécution et peu coûteux en place mémoire)

3.2.6. Le traitement des erreurs

Un compilateur doit détecter les erreurs lexicales, syntaxiques ou sémantiques, donner des diagnostics clairs et précis et être capable de poursuivre l'analyse du programme source.

Exemple d'erreurs

• Lexicales, comme l'écriture erronée d'un identificateur, d'un mot clé ou d'un opérateur ;
• Syntaxiques, comme une expression mathématique mal parenthésée ;
• Sémantiques, comme un opérateur appliqué à un opérande incompatible ;
• Logiques, comme un appel récursif infini.

Chapitre 2

GENERALITES SUR LES LANGAGES

1. Introduction

Les langages peuvent prendre des formes très diverses selon la nature de ce qu'ils expriment : langage naturel, langage mathématique, langage de programmation (Pascal, C...), langages formels...

Une première formalisation de La description des langages formels a été proposée par le linguiste **Noam CHOMSKY (1950)**. Un langage formel est constitué d'un vocabulaire et de règles de grammaires entièrement connus. La théorie des langages formels s'occupe soit de générer l'ensemble des phrases (mots) d'un langage soit de déterminer si une phrase appartient ou non au langage.

Remarque

L'analyse ou la génération de phrases est, dans ce contexte, purement syntaxique. Elle est indépendante de la signification des symboles utilisés et des mots produits et/ou reconnus.

2. Mots

Définition

Un **symbole** est une brique élémentaire, un atome.

Si $a \in A$ atome alors $\forall\ x, y \in A, a \neq x + y$.

Un **alphabet** A est un ensemble fini et non vide de symboles.

Un **mot m** (ou **chaîne**) sur un alphabet A est une suite finie $m_1 m_2 \cdots m_n$ de lettres de A. L'entier n est appelé la longueur du mot m, notée |m|.

Le **mot vide** est le mot de longueur 0, c'est-à-dire ne contenant aucun symbole. Il est souvent noté ε.

Remarque

Les **alphabets** seront décrits comme des ensembles de **symboles** (entre accolades), composés de plusieurs caractères. Les symboles représentés par des signes de ponctuation seront entre quottes.

Remarque

Soit l'alphabet {a, b, c, h, ch, ';', ','}.

ch et c + h ≡ ch sont théoriquement différents : ch (|ch| = 1) et c + h ≡ ch (|c + h| = 2).

Notation

• a, b, c, d, e, f, 0, 1... désignent des symboles.

• t, u, v, x, y, z désignent des chaînes.

• La chaîne aaaa est aussi notée a^4 (a^0=ε).

• Le symbole m(i) désigne le i-ème symbole de la chaîne m.

Remarque

L'opération particulière définie sur les mots est la **concaténation**. Soient x et y deux chaînes, la concaténation de x et y est xy.

N.B. La concaténation n'est pas commutative.

La chaîne vide ε est un élément neutre pour la concaténation x ε = ε x = x.

Définition

Étant donné un alphabet A :

- A* : ensemble de tous les mots sur A.
- Dans A*, une opération **concaténation** est définit.

u et v deux mots de longueur p et q, la concaténation de u et v (dans cet ordre) est le mot w de longueur p+q défini par : w(i) = u(i) pour $1 \leq i \leq p$ et w(p+i) = v(i) pour $1 \leq i \leq q$.

Remarque

- L'ensemble A* des mots ainsi construit est infini dénombrable.
- Les symboles de l'alphabet sont atomiques, ie : si $a \in A$, $\forall x,y \in A^+$, $a \neq xy$
- Il n'y a pas d'inverse pour l'opération de concaténation, ie : si $u \in A^+$, $\forall v \in A^+$, $uv \neq \varepsilon$

3. Théorème de la Décomposition

Tout mot x (de longueur n = |x|) sur un alphabet A se décompose de façon unique en $x_1 + x_2 + ... + x_n$, également noté $x_1x_2...x_n$, où $\forall i \in [1..n]$, $x_i \in A$.

Démonstration

Comme les symboles de l'alphabet sont des atomes, il n'est pas possible de diviser un mot plus qu'au niveau de ces symboles. Si deux divisions en symboles étaient possibles, cela voudrait dire qu'un symbole peut être divisé et donc il ne serait pas atomique.

Remarque

Une chaîne sur un alphabet A peut également être définie récursivement par :

- la chaîne vide ε est une chaîne sur A;
- si x est une chaîne sur A et a ∈ A alors la chaîne ax et xa sont des chaînes (mots) sur A (Attention, dans le cas général, ax ≠ xa);
- aucun autre objet n'est une chaîne sur A.

Notation

A^n : ensemble des mots (ou chaînes) de longueur n (pour n ∈ N) construits sur A

$(A^n = \{x_1x_2...x_n \mid \forall\ i,\ 1 \le i \le n,\ x_i \in A\})$

Par conséquent : toutes les chaînes *y* de l'ensemble A^3 ont pour longueur |y|=*3*.

A^+ : L'ensemble des chaînes de longueur finie et non nulle définies sur A.

$A^+ = \cup_{n=1..\infty} A^n = \{x / n \in N, n \ge 1 \text{ et } x \in A^n\}$

$A^* = A^+ \cup \{\varepsilon\}$

Par convention :

$A^0 = \varepsilon$

$A^* = \cup_{n=0..\infty} A^n$

4. Relations entre mots

Définition

Si x et y sont deux chaînes quelconques sur A, alors :

x est un **préfixe** de y (x ∈ Pref(y)) s'il existe z dans A* tel que y = xz, autrement dit x ∈ Pref(y) si et seulement si ∃ z ∈ A*, y = xz ;

x est un **suffixe** de y (x ∈ Suff(y)) s'il existe z dans A* / y = zx, autrement dit x ∈ Suff(y) si et seulement si ∃ z ∈ A*, y = zx ;

x est une **sous-chaîne** (ou **facteur**) de y s'il existe w et z dans A* tel que y = wxz, autrement dit x ∈ Fact(y) si et seulement si ∃ w, z ∈ A*, y = wxz ;

Si $x \neq y$ (c'est-à-dire si $z \neq \varepsilon$), alors le préfixe ou le suffixe est dit **propre**.

$Pref(x) \subset Fact(x)$ et $Suff(x) \subset Fact(x)$.

Définition

Une **occurrence** d'une chaîne x dans une chaîne y est une apparition de x à un endroit précis dans la chaîne y.

5. Relations d'ordre

Soit A* défini sur $A = \{x_1, x_2, ..., x_n\}$, il est possible de définir plusieurs relations d'ordre.

5. 1. Ordre préfixiel

Ordre partiel définit par : $u < v$ si et seulement si u est un préfixe propre de v.

5. 2. Ordre lexicographique

- Ordre total défini par : $u < v$ si et seulement si $u = wau_2$ et $v = wbv_2$ tels que $w \in A^*$, $a < b$ avec a et $b \in A$;

- $u<v$ si et seulement si $u = aw$, $v = bz$ et $(a < b)$ ou $((a = b)$ et $(w < z))$, $a, b \in A$ et $w, z \in A^*$ avec $\varepsilon < t \ \forall t \in A^+$. Tout ceci est valable si et seulement si $\forall x_i, x_j \in A$, $x_i \leq x_j$ si $i \leq j$.

5. 3. Ordre hiérarchique

Ordre total pour lequel les mots sont classés en premier lieu par longueur, puis pour les mots de même longueur, par ordre lexicographique.

6. Langages et opérations

Définition

Un langage L sur un alphabet A est un ensemble de chaînes (ensemble de mots) sur A. L est un sous-ensemble de A*, autrement dit $L \subset A^*$. L'ensemble des langages L sur A est l'ensemble $P(A^*)$ des parties de A^*, autrement dit : $L \subset P(A^*)$.

Définition

Étant donné un alphabet A, parmi tous les langages L de P(A*) :

Langage neutre : langage dont le seul mot est la chaîne vide, $L = \{\varepsilon\}$.

Langage vide : langage qui ne contient aucun mot, soit $L = \emptyset$. $\emptyset \neq \{\varepsilon\}$

Un ***langage L*** est dit posséder la propriété ***préfixe*** (resp. ***suffixe***) si aucune chaîne de L n'est préfixe propre (resp. suffixe propre) d'une autre chaîne de L.

Remarque

Comme les langages sont des ensembles, des opérations ensemblistes classiques peuvent être appliquées : union, intersection, complémentation, ... De plus, par extension de la concaténation des mots aux langages, quelques nouveaux opérateurs peuvent être définis.

Définition

Soit A un alphabet. Sur les langages de P(A*) les opérateurs suivants sont définis: soient L et M deux langages sur A,

6. 1. Opérateurs ensemblistes classiques

Union : $L \cup M = \{x \mid x \in L \text{ ou } x \in M\}$;

Intersection : $L \cap M = \{x \mid x \in L \text{ et } x \in M\}$;

Différence (ou exclusion) : $L\backslash M = L-M = \{x \mid x \in L \text{ et } x \notin M\}$;

Complémentaire sur A* : $Comp(L)= A^*\backslash L = \{x \mid x \in A^* \text{ et } x \notin L\}$;

6. 2. Opérateurs induits par la concaténation des mots

Produit des langages : $LM = L\times M = \{xy \mid x \in L \text{ et } y \in M\}$;

Fermeture de Kleene : $L^* = \cup_{i=0..\infty} L^i$ où $L^0 = \{\varepsilon\}$ et $L^n = LL^{n-1} = L^{n-1}L$;

Fermeture positive : $L^+= \cup_{i=1..\infty} L^i$.

Remarque

Le produit est parfois appelé "concaténation de deux langages". Cette concaténation est notée × (et le symbole × est souvent omis), mais il s'agit bien de deux concaténations différentes : l'une entre mots et l'autre entre ensembles de mots.

La concaténation de deux langages est l'ensemble des mots obtenus en concaténant un mot du premier langage avec un mot du second.

Propriété

- Le langage vide est absorbant pour la concaténation des langages : $\emptyset L = \emptyset = L\emptyset$
- $L^+ = LL^* = L^*L$
- $L^* = \{\varepsilon\} \cup L^+$
- $\emptyset^* = \{\varepsilon\}$
- $\{\varepsilon\}^* = \{\varepsilon\}$

Chapitre 3

LES LANGAGES RATIONNELS

1. Introduction

Parmi tous les langages constitués par les parties de A^* **engendrés** par un alphabet A, une classe particulière est distinguée : la classe *Rat(A*)* des *langages rationnels* sur A. L'intérêt de cette classe est qu'il existe une méthode simple permettant de décider pour chacun de ses langages si un mot lui appartient ou non : **Automates Finis**.

2. Langages rationnels

Définition

Soit A un alphabet. Les **langages rationnels sur A** (appelés aussi **langages réguliers**) sont les éléments de la classe Rat(A*) définie inductivement de la façon suivante : Rat(A*) est le plus petit sous-ensemble de P(A*) tel que :

1. $\emptyset \in Rat(A^*)$;
2. $\{\varepsilon\} \in Rat(A^*)$;
3. $\forall\ a \in A, \{a\} \in Rat(A^*)$;
4. Si $L_1 \in Rat(A^*)$ et $L_2 \in Rat(A^*)$ alors $L_1 \cup L_2 \in Rat(A^*)$;
5. Si $L_1 \in Rat(A^*)$ et $L_2 \in Rat(A^*)$ alors $L_1 \times L_2 \in Rat(A^*)$;
6. Si $L_1 \in Rat(A^*)$ alors $L_1^* \in Rat(A^*)$.

3. Théorème des parties finies

Toute partie finie L de A* est dans Rat(A*).

Démonstration

Comme toute partie finie peut être vue comme la réunion de singletons, L peut être obtenu par un certain nombre d'applications des points 1 à 5 et en utilisant le théorème de décomposition unique d'un mot.

Remarque

A l'aide des opérateurs de la définition précédente, de nombreux langages désignés par des expressions mathématiques peuvent être construits.

Remarque

Il existe pour les langages rationnels une écriture simplifiée, plus compacte et pratique appelée les expressions rationnelles. Chaque expression rationnelle décrit un langage rationnel précis.

4. Expressions rationnelles

Définition

Les **expressions rationnelles** sur A décrivent les **langages rationnels**. Elles sont définies de la façon suivante :

1. ∅ est une expression rationnelle qui décrit le langage rationnel ∅ ;

2. ε est une expression rationnelle qui décrit le langage rationnel {ε} ;

3. pour tout a de A, a est une expression rationnelle qui décrit le langage rationnel {a} ;

4. Si l_1 et l_2 sont des expressions rationnelles qui décrivent L_1 et $L_2 \in Rat(A^*)$ alors :

a. $(l_1 \mid l_2)$ et $(l_1 + l_2)$ sont des expressions rationnelles identiques qui décrivent le langage rationnel $L_1 \cup L_2 \in Rat(A^*)$,

b. $(l_1 l_2)$ et $(l_1.l_2)$ sont des expressions rationnelles identiques qui décrivent le langage rationnel $L_1 \times L_2 \in Rat(A^*)$,

c. $(l_1)^*$ est une expression rationnelle qui décrit le langage rationnel $(L_1)^*$.

Comme la réunion et la concaténation sont associatives, les parenthèses inutiles sont effacées. *(u | v | w)* remplacera *((u |v) | w) ou (u | (v | w))*.

N.B.

Il est sous-entendu qu'aucun des symboles "|", "*", "ε", "(" et ")" n'appartient à l'alphabet *A*. Sinon, il faut soit les différencier en renommant les symboles, soit les entourer de guillemets.

Théorème

Tout langage dénoté par une expression rationnelle est un langage rationnel.

Soit A un alphabet et C = { |, *, ε, (,)} tel que A ∩ C = ∅ ;

Un élément w est une expression rationnelle si et seulement si :

- $w \in A^*$;
- w est de la forme w_1^*, $w_1 w_2$ ou $w_1 \mid w_2$ avec w_1 et w_2 des expressions rationnelles.

Une expression rationnelle désigne un langage rationnel de la façon suivante :

- $w \in A^*$ désigne le langage dont le seul mot est w (w peut être ε) ;
- soient w_1 et w_2 des expressions rationnelles désignant les langages $L(w_1)$ et $L(w_2)$ alors:

- w_1w_2 désigne le langage dont chaque mot est obtenu en concaténant (juxtaposant) un mot du langage désigné par w_1 avec un mot du langage désigné par w_2 (dans l'ordre) : $L(w_1)\ L(w_2)$;
- w^* désigne le langage dont chaque mot est obtenu en concaténant un nombre quelconque (éventuellement nul) de mots du langage désigné par w : $(L(w))^*$;
- $w_1|w_2$ (parfois notée w_1+w_2) désigne la réunion des langages désignés par w_1 et w_2. Les mots de $w_1|w_2$ sont ceux qui appartiennent à w_1 ou à w_2 : $L(w_1) \cup L(w_2)$.

Notation

- $w^+ = w\ w^* = w^*\ w$;
- w? est une abréviation pour $w \mid \varepsilon$;
- la notation [a, b, c], où a, b et c sont des symboles, abrège l'expression rationnelle a|b|c. Une classe de caractères comme [a-z] abrège l'expression rationnelle a | b | ... | z.
- w^n, pour un $n \geq 0$ désigne la chaîne composée de n occurrences du mot du langage désigné par w. Si n est quelconque : $w^n\ (n \geq 0) \equiv w^*$; $w^n\ (n > 0) \equiv w^+$.

Notation

w^n : un langage rationnel quand n est un entier fixé, par contre, ce n'est pas toujours vrai dans le cas général, quand n est une variable.

En effet, le langage désigné par a^nb^n n'est pas rationnel : il s'agit des mots commençant par un certain nombre de a et suivis **du même nombre** de b. Ce langage ne peut pas être obtenu à partir des parties finies d'un alphabet A par simple application des opérations ×, U et *. Ceci illustre bien le fait que tout langage sur A n'est pas nécessairement rationnel.

Définition

Une expression rationnelle est dite **standard** si et seulement si les seuls opérateurs utilisés sont les opérateurs ×, ∪ et *.

Remarque

Il se peut que deux expressions différentes puissent décrire un même langage. a*a et aa* désignent toutes deux l'ensemble des chaînes de n a consécutifs pour n>0.

Définition

Deux expressions rationnelles w_1 et w_2 sont dites équivalentes (ou par abus de langage "égales"), noté "$w_1 \equiv w_2$" (ou "$w_1 = w_2$"), si elles décrivent le même langage rationnel.

Propriété

Soient u, v et w des expressions rationnelles :

Union

u|v = v|u (commutativité)

u|u = u (idempotence)

u|∅ = u (∅ : élément neutre)

u|(v|w) = (u|v)|w (associativité)

La mise à l'étoile

$u^{**} = u^*$ (idempotence)

∅* = ε

La mise à l'étoile et l'union

u* = u|u* (absorption)

La concaténation

• uε = εu = u (ε : élément neutre à gauche et à droite)

• u∅ = ∅u = ∅ (∅ : élément absorbant à gauche et à droite)

• u(vw) = (uv)w (associativité)

La concaténation et l'union

• (u|v)w = uw|vw (distributivité à droite de la concaténation par rapport à l'union)

• u(v|w) = uv|uw (distributivité à gauche de la concaténation par rapport à l'union)

Remarque

uw ≠ wu (la concaténation n'est pas commutative) ! sauf si u = w.

Définition

Un **facteur itérant** est une sous-chaîne non vide pouvant être "étoilée". Autrement dit, v est facteur itérant de m ∈ L si m = uvw, |v| ≠ 0, ∀ i, $uv^iw \in L$ ($uv^*w \subset L$).

Définition

Un **mot primitif** est un mot u tel que $u = v^n$ si n=1, c'est-à-dire s'il n'est pas puissance d'un autre mot que lui-même.

5. Définitions rationnelles

Si A est un alphabet de symboles de base, une **définition rationnelle** est une suite de n définitions (n ∈ N) de la forme $\mathbf{d_i} \rightarrow r_i$ (pour i ∈ {1,..., n}), où chaque $\mathbf{d_i}$ est un nom distinct, n'appartenant pas à (A ∪ {|, *, ε, (,)})*, et chaque r_i une expression rationnelle sur l'alphabet augmenté A ∪ {d_1, ..., d_{i-1}}.

Remarque

- Chaque membre droit (r_i) d'une règle ($\mathbf{d_i} \rightarrow r_i$) ne peut faire intervenir que les nouveaux symboles définis **antérieurement** (les $\mathbf{d_j}$ tels que $j<i$) ;
- Généralement, il n'y a pas de définition récursive, comme le serait par exemple la définition caractérisée par l'unique règle $\mathbf{X} \rightarrow a\mathbf{X}b|ab$, qui engendre pour **X** le langage $\{a^nb^n \mid n \in N\}$. En effet, si l'on autorisait l'écriture de définitions récursives, on pourrait caractériser des langages qui ne sont pas rationnels, comme par exemple $\{a^nb^n \mid n \in N\}$.

Une seule définition récursive, est autorisée : la récursivité droite, c'est-à-dire des définitions de la forme $\mathbf{X} \rightarrow x\mathbf{X}|y$ avec x et y des expressions rationnelles.

6. Systèmes d'équations rationnelles

Les définitions rationnelles permettent de proposer des expressions rationnelles plus lisibles. Cependant, il est parfois nécessaire d'obtenir une expression rationnelle ne comportant que des symboles du vocabulaire. Il faut donc alors essayer de trouver cette expression. La simple substitution n'est pas toujours suffisante, surtout en cas de récursivité. La solution consiste à passer par la résolution d'un système d'équations rationnelles. La transformation s'effectue en utilisant directement les définitions rationnelles. L'opérateur → devient =, l'opérateur | devient +, parfois ε devient 1 et les identificateurs deviennent des variables. La résolution du système ainsi obtenu donne l'expression rationnelle du langage.

Définition

Les **systèmes d'équations rationnelles** sont des ensembles d'équations dont les coefficients sont des expressions rationnelles.

Remarque : Les solutions ne sont pas uniques.

Définition

Un ensemble d'équations rationnelles d'indéterminés $D = \{X_1, ..., X_n\}$ est en forme standard si $\forall X_i \in D$, il y a une équation de la forme : $X_i = a_{i0} + a_{i1}X_1 + ... + a_{in}X_n$ avec a_{ij} des expressions rationnelles sur A telles que $A \cap D = \emptyset$.

Remarque

Il est possible d'avoir $a_{ij} = \emptyset$ quand il n'y a pas de terme correspondant à X_j dans l'équation de X_i. $\emptyset$ joue le rôle de 0 ; ε celui de 1 (l'équation pour X_i comporte alors X_j sans coefficient à droite).

6. 1. Lemme d'Arden

Soient K et L deux langages sur A* ($K \subseteq A^*$ et $L \subseteq A^*$), $\varepsilon \notin K$, alors :

1) K*L est l'unique solution de l'équation $X = KX + L$.

2) LK* est l'unique solution de l'équation X = XK + L.

Si $\varepsilon \in K$, alors A* est solution et K*L la plus petite solution.

De ce lemme, il est possible de démontrer un certain nombre d'**identités rationnelles** très utiles pour démontrer des équivalences d'expressions rationnelles ou pour simplifier des expressions.

En particulier, les identités suivantes :

$(a + b)^* = a^* (ba^*)^*$ (e_1)

$(a + b)^* = (a^*b)^* a^*$ (e_2)

$(ab)^* = 1 + a (ba)^* b$ (e_3)

6. 2. Algorithme de résolution

Soit un système de n équations rationnelles notées $X_1, ..., X_n$. Cette méthode se déroule en trois phases :

a) Écrire, quand c'est possible, les équations du système sous la forme : $X_i = x_iX_i + Y_i$ avec x_i expression rationnelle sur A et Y_i expression rationnelle de la forme $y_0 + y_1X_1 + ... + y_{i-1}X_{i-1} + y_{i+1}X_{i+1} + ... + y_nX_n$ où y_i des expressions rationnelles sur A.

b) Pour toutes les équations X_i de X_1 à X_{n-1}, sachant que selon le lemme d'Arden, l'unique solution pour $X_i = x_iX_i+Y_i$ est $x_i^*Y_i$, remplacer dans toutes les équations $X_{i+1}... X_n$ la variable X_i par $x_i^*Y_i$.

c) Pour toutes les équations X_i de X_n à X_1, calculer la valeur de X_i en appliquant le Lemme d'Arden et remplacer X_i par cette valeur dans toutes les équations de X_{i-1} à X_1.

Remarque

Il faut penser à simplifier continuellement les expressions rationnelles pour avoir un résultat le plus simple possible. De plus, en fonction des simplifications choisies et de l'ordre adopté, les résultats ne sont pas toujours identiques (du point de vue des expressions obtenues) mais toujours équivalents.

7. Lemme de l'étoile (de la pompe)

Soit L un langage rationnel sur un alphabet A. Alors il existe un entier naturel n tel que pour tout mot z de L vérifiant $|z| > n$, il existe u, v, w $\in$ A* tels que z = uvw, $v \neq \varepsilon$, $|uv| \leq n$ et pour tout $i \in N$, $uv^iw \in L$.

Attention

Ce lemme peut s'appliquer à des langages qui ne sont pas rationnels. Mais tout langage rationnel vérifie ce lemme. $Rat(A^*) \subset Etoile(A^*) \subset P(A^*)$

Remarque

1) Ce théorème permet souvent de **démontrer *par l'absurde* qu'un langage L n'est pas rationnel** : il suffit de montrer que, **pour tout n de N**, **il existe un mot** z de L qui ne vérifie pas la condition **pour un i donné**.
2) Par contre, **montrer qu'un langage vérifie le lemme de l'étoile ne prouve pas qu'il est rationnel**.

Exemple

Le langage $\{a^m b^m \mid m \in N\}$ est il rationnel?

Hypothèses : Il existe un entier positif $n \in N$, tel que $\forall\ z \in L$, $|z| > n$, (H1) $\exists$ u,v,w, tels que $z = uvw$, (H2) $v \neq \varepsilon$, (H3) $|uv| \leq n$, (H4) $\forall\ i \in N$, $uv^i w \in \{ a^m b^m \mid m \in N\}$

Considérons le mot $z = a^n b^n$ ($|z| = 2n$, donc $|z| > n$). Soient u, v et w tels $z = uvw$ (H1) et vérifiant H2, H3 et H4 (z est plus long que n et par hypothèse, u, v et w existent).

Tout préfixe de z de longueur $l \leq n$ est nécessairement composé uniquement de a. Comme uv est un préfixe de z (H1 : $z = uvw$) et $|uv| \leq n$ (H3), ceci est vrai en particulier de uv. Donc il existe k, $p \in N$ ($p \neq 0$) tels que $u = a^k$, $v = a^p$ (H2 : $v \neq \varepsilon$) et $k+p \leq n$.

On a $z = uvw = a^k a^p w$, avec $w = a^{n-(k+p)} b^n$ (car $z = a^n b^n$).

H4 s'écrit alors $\forall\ i \in N$, $a^k (a^p)^i a^{n-(k+p)} b^n \in \{a^m b^m \mid m \in N\}$,

or ceci est faux : $a^k (a^p)^i a^{n-(k+p)} b^n = a^k a^p (a^p)^{i-1} a^{n-(k+p)} b^n$

$$= a^k a^p a^{n-(k+p)} (a^p)^{i-1} b^n = a^n (a^p)^{i-1} b^n$$

comme $p \neq 0$, pour n'importe quel $i > 1$, $a^n (a^p)^{i-1} b^n \notin L$.

Donc l'hypothèse est fausse et d'après le théorème de l'étoile $\{a^m b^m \mid m \in N\}$ n'est pas un langage rationnel.

Chapitre 4

LES GRAMMAIRES DE CHOMSKY

1. Introduction

Un langage est défini par l'ensemble de ses mots et de ses phrases correctes. Il est impossible de donner la liste des phrases (ou mots) correctes d'un langage parce qu'elle est souvent infinie. Pour les langages rationnels nous pouvons les dénoter par des expressions rationnelles. Toutefois, tous les langages ne sont pas rationnels et cette solution n'est donc pas toujours possible.

Pour atteindre d'autres classes de langage, divers types de grammaires sont utilisés. Les grammaires peuvent être vues comme des "algorithmes" d'une part pour construire les mots du langage et d'autre part pour vérifier la validité d'un mot par rapport à un langage.

2. Structures d'une grammaire

Une grammaire à structure de phrase G permet de caractériser un langage L(G), qui est un ensemble de phrases (ou mots), appelé le langage engendré par la grammaire. Une telle grammaire ressemble à une définition rationnelle.

1. $\mathbf{V_T}$: vocabulaire terminal dont chaque élément est appelé symbole terminal. Ce vocabulaire correspond à l'alphabet pour les langages rationnels et les symboles terminaux aux symboles de l'alphabet. $L(G) \subseteq V_T^*$. Dans ce cours, ne seront considérer que les vocabulaires terminaux composés de symboles (lettres, chiffres, caractères spéciaux comme '+', '-', ';'...). En **compilation**, ce vocabulaire est un ensemble d'unités lexicales (identificateurs réservés ou non, paramètres, nom, opérateurs...).

2. $\mathbf{V_N}$: vocabulaire non terminal dont chaque élément est appelé symbole non terminal. Les symboles non terminaux peuvent être vus comme les nouveaux symboles des définitions rationnelles (utilisés comme membres gauches des règles). Ils ont toutefois un rôle un peu plus large. Ils n'appartiennent pas au vocabulaire de base (terminaux) et sont destinés à être éliminés, remplacés pour former les mots du langage.

3. Des **règles** permettant de déterminer quelles séquences de V_T^* sont légales. Ces règles ressemblent à celles des définitions rationnelles mais sont plus souples et plus générales.

Définition

Une **grammaire formelle**, appelée aussi **grammaire de Chomsky**, G est une description de la forme des symboles et des phrases d'un langage noté L(G). Elle est définie par un quadruplet $G = (V_T, V_N, S, R)$ où :

a) V_T est un ensemble fini non vide : le vocabulaire terminal ou alphabet; $L(G) \subseteq V_T^*$.

b) V_N est un ensemble fini non vide : le vocabulaire non terminal (on parle aussi de variable ou de catégorie syntaxique) ;

$V_T \cap V_N = \emptyset$; $V = V_T \cup V_N$: le vocabulaire de la grammaire;

c) $S \in V_N$ est un symbole non terminal particulier appelé l'axiome (symbole de départ);

d) R est un ensemble de **règles**, appelées **règles de production** (ou **règles de réécriture**), de la forme $\alpha \rightarrow \beta$ avec $\alpha \in V^+$ et $\beta \in V^*$.

Remarque

Des règles de la forme $\alpha \rightarrow \beta_1 \mid \beta_2 \mid ... \mid \beta_n$ avec $n \geq 1$ et pour tout $1 \leq i \leq n$, $\beta_i \in V^*$ peuvent être écrites pour abréger n règles ayant α en partie gauche, de la forme $\alpha \rightarrow \beta_i$.

Définition

Les formes sententielles sont les mots atteints en prenant l'axiome S comme mot de départ et en appliquant les règles un nombre fini de fois. Les suites d'applications de règles sont appelés "dérivations".

Définition

Les mots du langage caractérisé par la grammaire sont les mots accessibles qui ne sont composés que de symboles terminaux.

Définition

Soit $G = (V_T, V_N, S, R)$ une grammaire et $V = V_T \cup V_N$, alors l'ensemble des **formes sententielles** (ou **proto-phrases**) de G est défini récursivement par :

- L'axiome "S" est une **forme sententielle** de G.
- Pour tous $v, x, y, z \in V^*$, si "xyz" est une forme sententielle de G et si $y \rightarrow v \in R$, alors "xvz" est une forme sententielle de G.

Le **langage engendré** par G est $L(G)=\{x \in V^* \mid x \text{ est une forme sententielle de G et } x \in V_T^*\}$.

Une grammaire G est peut être utilisée soit pour générer les phrases du langage L(G) soit pour déterminer si une phrase donnée appartient ou non au langage L(G) :

• La génération consiste à produire des chaînes de symboles par application des règles de la grammaire à partir de son axiome S.

• La vérification (l'analyse) consiste à retrouver la succession d'applications de règles qui a permis d'obtenir un mot en partant de l'axiome S.

Définition

Soit G = (V_T, V_N, S, R) une grammaire : La relation «**dérive directement de**» sur V* notée → est définie par : ∀ z, z' ∈V*, z → z' ↔ ∃ t, u, v, w ∈ V*, z = uvw et z' = utw et v→ t ∈ R.

Si u ∈ V_T* (resp. w ∈ V_T*) alors uvw → utw est une **dérivation à gauche** (resp. **à droite**). Intuitivement, dans uvw le non terminal v le plus à gauche (resp. à droite) est remplacé pour obtenir utw.

Notation

=R→ pour désigner explicitement la règle utilisée.

a =n→b désigne une dérivation de longueur n.

a =*→b désigne une dérivation de longueur quelconque, nous disons alors que a **engendre** b ou, symétriquement, que b **dérive de** a.

Définition

Deux grammaires G_1 et G_2 sont dites équivalentes si $L(G_1) = L(G_2)$.

3. Classification des grammaires

3. 1. Type 0 : sans contraintes

x → y avec x ∈ V* × V_N × V* et y ∈ V*.

3. 2. Type 1 : contextuelle (sensible au contexte)

- x → y avec |x|≤|y|.

Les productions sont soumises à la contrainte que la partie droite de la règle doit contenir au moins autant de symbole que la partie gauche. Le cas particulier S → ε est permis si S n'apparaît pas dans la partie droite d'une règle de G.

- xAy → xBy avec x, y ∈ V*, A ∈ V_N et B ∈ V^+ tel que |B| ≥ 1.

Les grammaires de type 1 sont dites sensibles au contexte car leur définition permet le remplacement de A par B uniquement dans le contexte précis où A est entouré par les symboles x et y.

3. 3. Type 2 : hors-contexte (algébrique)

X → x avec X ∈ V_N et x ∈ V^+.

Le membre gauche de la production est un unique symbole non terminal qui peut être réécrit par une chaîne quelconque du vocabulaire de la grammaire.

3. 4. Type 3 : linéaires (régulières à droite (resp. à gauche))

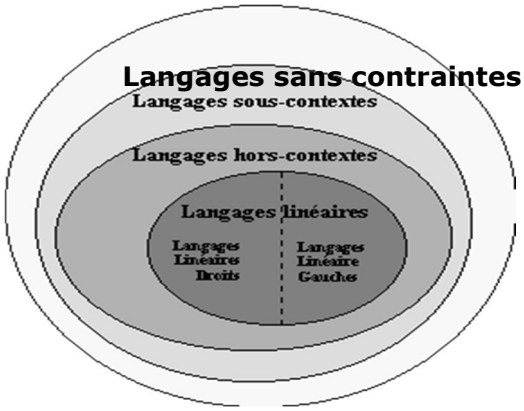
$X \rightarrow x_1 \mid x_2Y$: linéaire à droite et $X \rightarrow x_1 \mid Yx_2$: linéaire à gauche avec $X, Y \in V_N$ et $x_1 \in V_T^+$, $x_2 \in V_T^*$.

Le membre gauche de la production est un unique non terminal et le membre de droite est soit une chaîne de terminaux soit une chaîne de terminaux suivie par un seul non terminal.

3. 5. Grammaire de Type 4

La restriction des grammaires de type 4 interdit la présence d'un non-terminal à droite de la flèche. Ces grammaires représentent des langages finis.
$X \rightarrow x_1$ avec $X \in V_N$ et $x_1 \in V_T^+$

Ces classes de langage satisfont la relation d'inclusion stricte suivante :



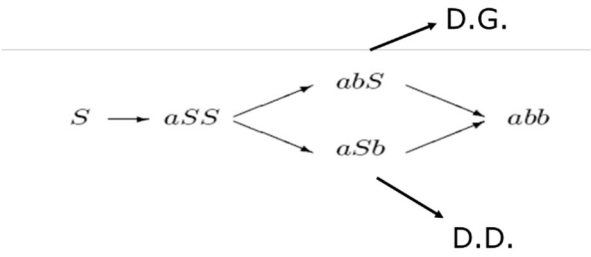
$cl_4 \subset cl_3 \subset cl_2 \subset cl_1 \subset cl_0$

4. Dérivations à gauche et à droite

Lorsqu'un mot est engendré par une grammaire, il peut l'être de plusieurs façons. Par exemple dans la grammaire :

$$G : \begin{cases} S \rightarrow aSS \\ S \rightarrow b \end{cases}$$

Le mot abb peut être obtenu par les deux dérivations :



5. Cas des Langages Rationnels

Définition

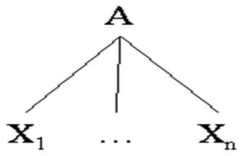
Un langage est linéaire s'il peut être engendré par une grammaire linéaire. Un langage est rationnel si et seulement s'il peut être engendré par une grammaire linéaire gauche (ou droite)

6. Représentation des grammaires type 2 avec la norme BNF ("Backus-Naur-Form")

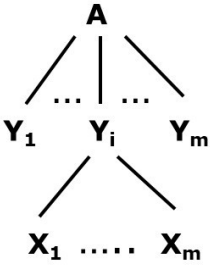
Les symboles non terminaux sont encadrés par des guillemets ("..." ou <<...>>), les symboles terminaux ne sont pas distingués (parfois les non-terminaux sont en majuscule et les terminaux en minuscule). L'axiome est la partie gauche de la première règle énoncée. La disjonction est notée par une barre verticale (|). La flèche des règles est remplacée par le signe "::=".

7. Arbres de dérivations

L'idée consiste à associer à chaque règle R : $A \rightarrow X_1 \ldots X_n$ de la grammaire, où $A \in V_N$ et $\forall\ 1 \leq i \leq n$, $X_i \in V$, la structure d'arbre suivante :



Une dérivation de la forme $A =n \rightarrow Y_1 \ldots Y_{i-1}\, Y_i\, Y_{i+1} \ldots Y_m = R \rightarrow Y_1 \ldots Y_{i-1}\, X_1 \ldots X_m\, Y_{i+1} \ldots Y_m$ est représentée par l'arbre associé à $A =n \rightarrow Y_1 \ldots Y_m$, dans lequel nous remplaçons le symbole non terminal correspondant à Y_i par l'arbre associé à R.



Une chaîne du langage est formée en parcourant les flèches et en recueillant les symboles rencontrés.

Remarque

Un arbre de dérivation d'un mot du langage engendré par une grammaire représente une façon de dériver ce mot à partir de l'axiome. La racine de l'arbre est l'axiome, ses feuilles sont les symboles (terminaux) du mot, et ses nœuds internes représentent les applications de règles. Une chaîne du langage est formée en parcourant les flèches et en recueillant les symboles rencontrés.

7.1. Ambiguïté

Une grammaire est dite ambiguë si au moins un mot dans son langage a plus d'un arbre de dérivation.

Chapitre 5

LES AUTOMATES D'ÉTAT FINI

1. Introduction

Les automates sont des objets mathématiques, très utilisés en informatique, qui permettent de modéliser un grand nombre de systèmes (informatiques). En compilation les automates servent à constituer des analyseurs lexicaux qui permettent de repérer les mots d'un langage. De façon très informelle, un automate est un ensemble "d'états du système", reliés entre eux par des "transitions" qui sont marquées par des symboles. Étant donné un "mot" fourni en entrée, l'automate lit les symboles du mot un par un et va d'état en état selon les transitions. Le mot lu est soit accepté par l'automate soit rejeté.

2. Automates finis

Définition

Un **automate fini** (**AFN**) est défini par un quintuplé (A, Q, I, F, μ) tel que :

A est **l'alphabet** (vocabulaire), ensemble fini, non vide de symboles ;

Q est **l'ensemble des états** possibles pour la machine (fini et non vide) ;

$I \subseteq Q$ est l'ensemble des états initiaux ou états de départ ;

$F \subseteq Q$ correspond aux états finaux ou états d'acceptation ;

μ la fonction de transition telle que :

$\mu : Q \times A \cup \{\varepsilon\} \rightarrow Q$.

Pour la configuration courante d'état et de symbole, $\mu(q_i, a_j) = q_k$ signifie que nous passons de l'état q_i vers l'état q_k en lisant le symbole a_j.

2.1 Fonctionnement d'un automate fini déterministe

L'automate prend en entrée un mot et l'accepte ou le rejette. Autrement dit, il le reconnaît ou ne le reconnaît pas. *Le langage associé à un automate est constitué de l'ensemble des mots qu'il reconnaît*. Voici comment l'automate procède pour décider si un mot appartient à son langage.

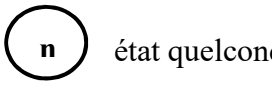
- Le processus commence à l'état de départ q_0.
- Les symboles du mot sont lus les uns après les autres.

• À la lecture de chaque symbole, la fonction de transition δ est employée pour se déplacer vers le prochain état (en utilisant l'état actuel et le caractère qui vient d'être lu).

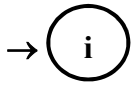
• le mot est reconnu si et seulement si le dernier état (*i.e.*, l'état correspondant à la lecture du dernier caractère du mot) est un état de *F*.

2.2. Représentation graphique

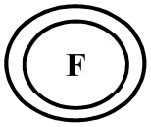
Un AFN peut être représenté par un graphe orienté, appelé "diagramme de transition" ou "graphe de transition". Les nœuds correspondent aux états et les arcs représentent les transitions possibles (calculées par la fonction de transition) entre les états. Ces arcs sont étiquetés par le symbole permettant cette transition. Les états initiaux sont représentés par un nœud possédant une flèche entrante sans origine. Un état final est représenté par un nœud doublement cerclé ou un nœud avec une flèche sortante.

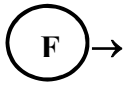


état quelconque

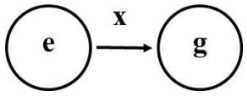


état initial





état final



est une transition où e est l'état de départ et f l'état d'arrivée ((x,e) → f). Cette notation est équivalente à μ(e, x) = f (ou δ(e, x) = f). Nous disons que l'automate passe de l'état **e** à l'état **f** en lisant le caractère **x**.

2.3. Représentation matricielle

Un automate peut être également caractérisé par une matrice de transition. En effet, la fonction de transition est très souvent représentée par une "table de transition" (matrice de transition) qui sont des tableaux (ou des matrices) dont les colonnes correspondent aux symboles et les lignes aux états. La cellule (l_i,c_j) indique les transitions (a_i, q_j) → q_k.

Exemple

L'automate T étudié précédemment peut être représenté par :

μ	**a**	**b**
→ 1	**1**	**2**
*** 2**	**1**	**2**

Il est à noter que dans la représentation matricielle, devant un état initial doit figurer une flèche et devant un état final une étoile.

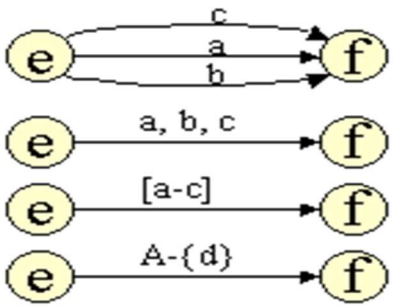
Définition

Le langage reconnu (ou accepté) par un automate fini non déterministe (AFN) est l'ensemble des mots acceptés par l'automate. Un mot w est reconnu (accepté) par un AFN si :

- Il existe un chemin menant d'un état initial à un état final, étiqueté par la suite des symboles de w.
- ε est neutre pour la concaténation.

Remarque

Dans certains cas, il est plus facile de représenter un groupe de transitions par une notation ensembliste. Par exemple sur l'alphabet A = {a, b, c, d}, supposons que pour un état donné e, nous avons un passage à l'état f pour les symboles a, b et c.



3. Propriétés des états

Outre les états finaux et les états initiaux, il existe d'autres catégories d'états spécifiques : les états stériles, accessibles, co-accessibles, utiles, inutiles et isolés.

3. 1. État stérile

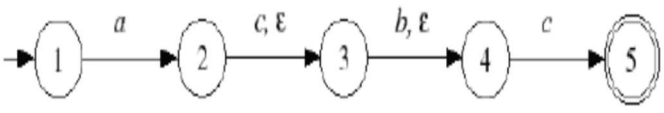
Un état stérile (état mort) est un état pour lequel il n'est pas possible d'atteindre un état final.

3. 2. État accessible

Un état q de A est dit accessible s'il existe un chemin partant de l'état initial et arrivant en q.

Remarque

Une ε-transition (notée ε sur l'arc d'un automate) permet de passer d'un état à l'autre d'un automate sans lire de symbole.



Le langage reconnu est L = {acbc, acc, abc, ac}

3. 3. État co-accessible

Un état q de A est dit co-accessible si de q part un chemin qui peut atteindre un état final.

3. 4. État utile

Un état q de A est dit utile s'il est à la fois accessible et co-accessible.

3. 5. État inutile

Un état inutile est soit un état qui n'est pas accessible, ou qui n'est pas co-accessible.

3. 6. État isolé

Un état e est dit **isolé** s'il ne participe à aucune transition.

4. Automate fini non déterministe

Un automate fini non-déterministe est un automate tel que dans un état donné, il peut y avoir plusieurs transitions avec le même symbole. Le fonctionnement d'un tel automate n'est donc pas totalement « déterminé », car on ne sait pas quel état l'automate va choisir.

5. Automate fini déterministe

Définition

Un automate fini déterministe (AFD) sur un alphabet A est un quadruplet (Q, δ, i_0, F) ssi :

1. δ est une fonction de transition : $\delta : Q \times A \rightarrow Q$.
2. avec un unique état initial.
3. à partir de chaque état, il y a au plus une transition d'étiquette donnée.
4. les ε-transitions ne sont pas autorisées.

5.1 Déterminisation d'un automate sans ε-transitions

Théorème

Soit T = (A, Q, I, F, δ) un AFN qui reconnaît le langage L = L(T) alors il existe un AFD T' = (A, Q', I', F', δ ') tel que L' = L(T') = L.

L'automate T'est construit de la façon suivante :

- Q' ⊆ P(Q), c'est-à-dire que les états de T' sont des ensembles d'états de T ;
- I' = {I} ;

- $F' = \{Q' \subseteq Q : Q' \cap F \neq \emptyset\}$;
- $\forall S \subset Q$ et $\forall a \in A$, $\delta'(a,S)=S'$ où $S'=\{p \mid \exists q \in S : p \in \delta(a,q)\}$.

5.1. Algorithme

1. Commencer avec l'état initial (ou ensemble des états initiaux)

2. Rajouter dans la table de transition tout les nouveaux "états" produits avec leurs transitions.

3. Recommencer en 2 jusqu'à ce qu'il n'y ait plus de nouvel "état" .

4. Tous les "états" contenant au moins un état terminal deviennent terminaux.

5. Renuméroter les états (facultatif)

Théorème

Si L et L' sont des langages reconnus par un AFN (respectivement un AFD), alors il en est de même :

- du complémentaire $A^* \setminus L$.
- de l'intersection $L \cap L'$.
- de l'union $L \cup L'$.
- de la concaténation LL'.
- de la mise à l'étoile L^*.

6. Minimisation d'un automate fini par raffinement successif

Certains AFD possèdent un nombre d'états et de transitions important. États et transitions ne sont pas toujours indispensables. Une première approche de cette minimisation peut consister à étudier les caractéristiques des états et de supprimer ceux qui sont inutiles.

Algorithme

Etape 0 :

$p, q \in Q$, $p \approx_0 q$ ssi

$(p \in F$ et $q \in F)$ ou $(p \notin F$ et $q \notin F)$

Etape i :

$\forall i>0$, $p \approx_i q$ ssi

$p \approx_{i-1} q$ et $\forall a \in A$, $\mu(p, a) \approx_{i-1} \mu(q, a)$

Etape finale (arrêt des itérations) :

$\approx_i = \approx_{i-1}$

Une division de l'ensemble des états en deux classes : les états terminaux et les autres.

Pour chaque lettre de l'alphabet, on examine si elle sépare deux états de la même classe, c'est-à-dire si sa transition envoie le premier dans une classe et le second dans une autre: si tel est le cas, on subdivise la classe en question.

On répète l'opération jusqu'à ce qu'aucune subdivision ne se produise plus.

La démarche

1) On construit un tableau de taille N × N (N le nombre d'états de l'automate). Chaque case dans ce tableau correspond à un couple d'états (p,q). Pour ne pas avoir deux fois le même couple ((p,q) et (q,p)) ni les couples inutiles (p,p), on garde seulement la partie triangulaire supérieure (ou inférieure) du tableau. Notre but final est de marquer toute les cases (p,q) pour des états p et q non-équivalents.

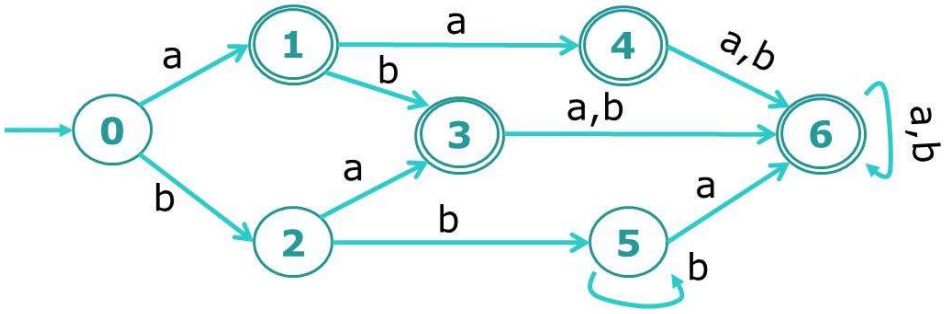
2) Au début on marque chaque case (p,q) correspondant à un état final et à un état non-final (c-à-d $p \notin F$ et $q \in F$ ou à l'envers, $p \in F$ et $q \notin F$).

3) On parcourt toutes les cases vides du tableau d'une manière quelconque en le modifiant de façon suivante : pour chaque case (p,q) et lettre a, on calcule $\mu(p,a) = p_0$ et $\mu(q,a) = q_0$.

On marque (p,q) si et seulement si la case (p_0,q_0) est déjà marquée. Le balayage est répété jusqu'au moment quand le tableau ne se modifie plus.

4) La relation d'équivalence se cache dans le tableau final : deux états p et q sont équivalents si et seulement si la case (p,q) n'est pas marquée.

Exemple



A = {0, 2, 5} B = {1, 3, 4, 6}

0							
1	X						
2		X					
3	X		X				
4	X		X				
5		X		X	X		
6	X		X			X	
	0	1	2	3	4	5	6

Initialisation X ≈ Itération N° 1



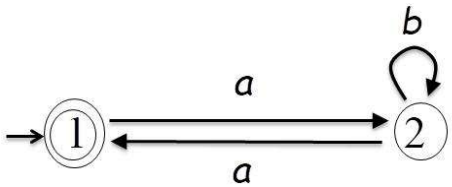
7. Automates finis et expressions régulières

Soit T = {A, Q, I, F, μ} un automate fini. Pour p ∈ Q, X_p est le langage des mots acceptés avec p comme état initial. Pour tout état X_p nous avons :

$$Xp = \begin{cases} \sum_{(p,a,q)\in\mu} aXq + \varepsilon & si\ p\ \in F \\ \sum_{(p,a,q)\in\mu} aXq & sinon \end{cases}$$

En appliquant cette équation pour chacun des états de l'automate, nous obtenons alors un système de n équations rationnelles (n = |Q|) à n inconnues où les inconnues sont les Xp et les coefficients sont des expressions rationnelles. La résolution d'un tel système s'effectue en utilisant principalement le lemme d'Arden. Le langage reconnu par l'automate est celui de l'état initial.

Exemple



$L_1 = aL_2 + 1$ (car l'état 1 est final)

$L_2 = bL_2 + aL_1$

$L_2 = b*aL_1$

$L_1 = ab*aL_1 + 1 = (ab*a)*$

L(T) est décrit par (ab*a)*.

Chapitre 6

ANALYSES LEXICALE, SYNTAXIQUE ET SEMANTIQUE

Partie I - Analyse lexicale

1. Introduction

Les éléments rencontrés dans un programme :

- Identificateur : nom du programme, X, Y, …
- Mots clés : si, sinon, tant que, fin, …
- Constantes : 12, 375, …
- Séparateur : +, -, ., >, <, …
- Commentaires : { }, * *, /* */, …

Le rôle de l'analyse lexicale (le « lexer» ou « scanner») est de reconnaître les caractères du programme, de les regrouper pour former des unités lexicales et d'éliminer les caractères inutiles tels que les commentaires, les espaces, les tabulations … Elle transforme le texte en une suite de lexèmes (token en anglais). Le programme est donc découpé en morceaux (identificateurs, entiers littéraux, ponctuation)

Flot d'entrée est le texte source, caractère par caractère.

Flot de sortie est l'ensemble des unités lexicales ou lexèmes (« tokens »).

Unité lexicale : désigne l'ensemble des lexèmes ayant la même signification grammaticale.

Exemple

Soit l'instruction suivante :
if (x1 * x2 < 1.0) {
y = x1 ;}

Le flot d'entrée est le suivant :



Après l'analyse lexicale, le flot de sortie est :

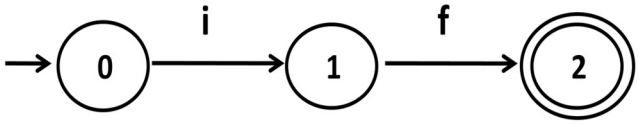
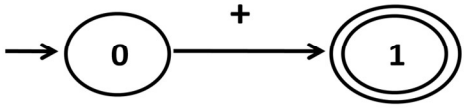
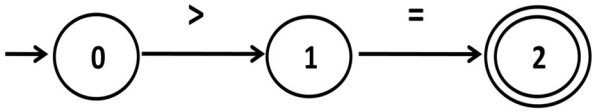


2. Structure d'un analyseur lexical

L'analyseur lexical est basé sur les expressions régulières. Il est donc un automate fini déterministe qui reconnaît l'union des langages des lexèmes avec un état terminal correspondant à la reconnaissance d'un mot d'un des langages des lexèmes.

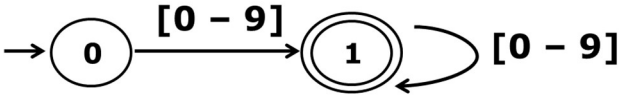
Il utilise un programme squelette commun à tous les analyseurs lexicaux et une table dans laquelle est stockée la fonction de transition de l'automate associé.

2. Reconnaissance des lexèmes

Lexème	Expression régulière	automate
Mot clé **if**	**if**	
Symbole +	+	
Symbole >=	>=	

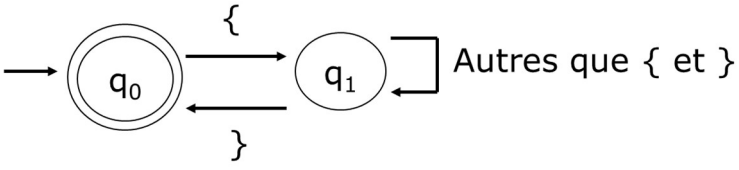
Constantes entières

Ces constantes sont représentées par l'expression régulière suivante : [0 - 9][0 – 9]*



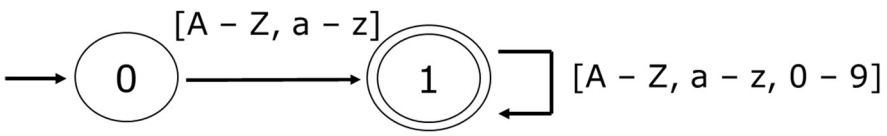
Commentaires

Un automate qui reconnait les commentaires.



Identificateurs

Les identificateurs sont représentés par l'expression régulière suivante : [A – Z, a - z][A – Z , a – z , 0 - 9]*



4. Reconnaissance contextuelle

Il existe des langages où une forme lexicale peut correspondre à différents lexèmes selon le contexte.

Remarque

Nous avons donc intérêt à ne faire reconnaître par l'automate que les identificateurs, puis à regarder dans une table si l'identificateur reconnu correspond à un mot clé. La recherche dans cette table peut être rapide (par dichotomie, par adressage dispersé . . .)

5. Génération d'analyseurs

Un analyseur lexical est un automate fini pour la réunion de toutes les expressions régulières définissant les lexèmes. Pour produire un analyseur à partir d'une spécification, il faut passer par les étapes suivantes :

- Traduction d'expressions rationnelles en automates finis non-déterministes ;
- Déterminisation ;
- Minimisation ;
- Génération directe de l'automate fini déterministe minimal.

Programme 1

Char ← prochain caractère
Etat ← q_0
Tant que (char ≠ EOF)
Etat ← δ (Etat, char)
Char ← prochain caractère
Si (Etat ∈ Q_F)
Alors accepter
Sinon rejeter

Remarque

Ces analyseurs passent beaucoup de temps à tester et à manipuler la variable d'état. Ce temps peut être optimisé en utilisant les analyseurs lexicaux codés directement.

6. Analyseur lexical codé directement

Soit l'automate A cité ci-dessus, l'analyseur lexical sera codé comme :

q_0 : Char ← prochain caractère
si (char = 'a')
alors goto q_1
sinon goto q_r

q_1 : Char ← prochain caractère
si (char = 'b' | char = 'c')
alors goto q_1
sinon si (char = 'EOF')
alors accepter
sinon goto q_r

q_r : rejeter

Remarque

Cette implémentation devient rapidement difficile à gérer manuellement mais une fois automatisé sera plus rapide que l'analyseur à base de table.

Conclusion

L'analyseur lexical prépare également le terrain pour l'analyse syntaxique. Il peut reconnaître les mots clés du langage. Pour cela, il y a deux possibilités :

1) Soit une suite explicite d'états reconnaissant chaque mot clé doit être introduite.
2) Soit, les identificateurs doivent être reconnus et comparés à une table des symboles contenant les mots clés du langage.

Partie 2 - Analyse syntaxique

1. Introduction

L'analyseur syntaxique analyse la suite de symboles issus de l'analyseur lexical et vérifie si cette suite de symboles est conforme à la syntaxe du langage définie par les règles de Backus-Naur. Pour cela, l'analyseur syntaxique essaye de construire l'arbre syntaxique correspondant au code. Dans cet arbre, les feuilles correspondent aux symboles issus de l'analyse lexicale et les nœuds intermédiaires correspondent aux objets grammaticaux.

Remarque

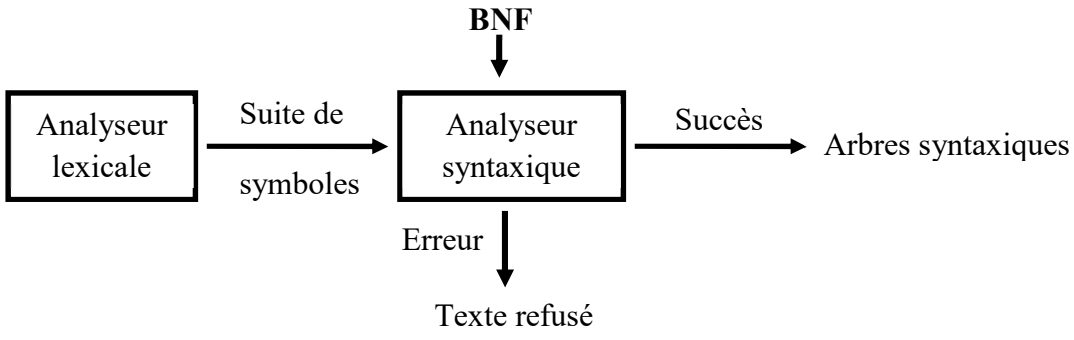
Si l'analyseur syntaxique ne parvient pas à construire l'arbre syntaxique du programme compilé alors la syntaxe du programme est erronée.

Définition

Soient $G = (V_T, V_N, S, P)$ et $x \in V_T^*$. L'analyse syntaxique est le processus d'établissement d'une correspondance entre l'axiome S et une chaîne terminale x :

1) $x \in L(G)$ ➔ construction de l'arbre syntaxique associé à la chaîne terminale.

2) $x \notin L(G)$ ➔ chaîne refusée.



Analyseur syntaxique

Conclusion

La construction des arbres syntaxiques fait apparaître la nécessité d'avoir défini la grammaire du langage sans ambigüité afin qu'à aucun moment, une suite de symboles issue de l'analyse lexicale puisse correspondre à plusieurs règles de production différentes, ce qui conduirait à une interprétation du sens différente à celle voulue par le programmeur.

Exemple de grammaire ambigüe

Soit $G = (V_T, V_N, S, P)$, V_T = {if, then, else} , V_N = {Bool, Instr}, S = Instr ,

P : Instr ➔ if bool then instr else instr

Instr ➔ if bool then instr

En se basant sur cette grammaire, la phrase : «**if bool then if bool then instr else instr**» est-elle ambiguë?

La phrase admet deux arbres syntaxiques différents. Le concepteur du langage veut que l'instruction soit interprétée comme le 2ème cas. D'où la nécessité de supprimer l'ambigüité en changeant les productions.

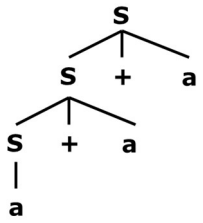
Définition

- Analyseur syntaxique non déterministe : aucun critère n'est imposé pour choisir la règle de production à appliquer.
- Analyseur syntaxique déterministe : des critères sont imposés pour choisir sans ambigüité la règle de production à appliquer.
- Analyseur syntaxique unique : pour une chaîne terminale donnée, un seul arbre syntaxique est fourni.
- Analyseur syntaxique multiple : plusieurs arbres syntaxiques possibles pour une même chaîne terminale donnée.
- Analyse syntaxique descendante : L'analyseur cherche, à partir de l'axiome et à l'aide des règles de la grammaire, à former la phrase donnée.
- Analyse syntaxique ascendante : L'analyseur cherche, à partir de la phrase et à l'aide des règles de la grammaire, à remonter à l'axiome.

Exemple

G = ({a, b, '+'}, {S}, S, R) , R: S $\rightarrow$ S + a | S + b | a | b, Soit x = a + b + b

L'analyseur syntaxique essayera de construire l'arbre de dérivation correspondant. L'analyse syntaxique descendante construit par exemple l'arbre suivant :



Les règles sont appliquées au hasard mais le résultat est erroné. La dernière règle est remplacée par une autre. Si le résultat est toujours faux, la règle précédente est analysée. Le concepteur devra donc choisir des analyseurs syntaxiques déterministes, uniques et descendants. D'où le choix des grammaires LL(K). En effet, il existe une classe de grammaire adaptée à l'analyse descendante déterministe : la classe des grammaires LL(k). Le premier L (Left) signifie que l'analyse des phrases se fait de la gauche vers la droite. Le deuxième L indique qu'il faut effectuer des dérivations à gauche avec une inspection de k symboles terminaux en avance sur le symbole courant traité.

La réalisation d'analyseurs syntaxiques pour des grammaires LL(k) avec k>1 est difficile et ces grammaires ne sont pas performantes. Les grammaires de type LL(0) sont trop restrictives. Par conséquent le type LL(1) est le plus utilisé. Le but de cette classe de grammaires est de toujours connaître la règle de production à appliquer et ceci en se basant uniquement sur le terminal suivant non encore traité de la phrase en cours d'examen.

3. **Calcul de PREMIER**

Pour toute chaîne α composée de symboles terminaux et non-terminaux, on cherche PREMIER(α) :

PREMIER(α) est l'ensemble de tous les terminaux qui peuvent commencer une chaîne qui se dérive de α. On cherche toutes les lettres a telle qu'il existe une dérivation $\alpha =^* \rightarrow a\beta$ ($\beta \in V^*$)

PREMIER(α) = $\{a \in V_T / \exists \beta \in V^*, \alpha^* \rightarrow a\beta\} \cup \{\varepsilon \text{ si } \alpha^* \rightarrow \varepsilon\}$

Exemple

$$\begin{cases} S \rightarrow Ba \\ B \rightarrow cP \mid bP \mid P \mid \varepsilon \\ P \rightarrow dS \end{cases}$$

$S \ ^* \rightarrow a$ alors $a \in$ PREMIER(S).

$S \ ^* \rightarrow cPa$ alors $c \in$ PREMIER(S).

$S \ ^* \rightarrow bPa$ alors $b \in$ PREMIER(S).

$S \ ^* \rightarrow dSa$ alors $d \in$ PREMIER(S).

S n'engendre pas le mot vide d'où PREMIER(S) = {a, b, c, d}

2.1. Calcul de Premier pour un symbole

1. Si X est un non terminal et $X \rightarrow Y_1Y_2\ldots Y_n$ est une production de la grammaire (avec Y_i symbole terminal ou non-terminal) alors :

i) ajouter les éléments de PREMIER(Y_1) sauf ε dans PREMIER(X).

ii) S'il existe j ($j \in \{2, \ldots, n\}$) tel que pour tout i = 1, …, j-1 on a $\varepsilon \in$ PREMIER(Y_i), alors ajouter les éléments de PREMIER(Y_j) sauf ε dans PREMIER(X).

iii) Si pour tout i = 1, …, n, $\varepsilon \in$ PREMIER(Y_i), alors ajouter ε dans PREMIER(X).

2. Si X est un non terminal et $X \rightarrow \varepsilon$ est une production alors ajouter ε dans PREMIER(X).

3. Si X est un terminal alors PREMIER(X) = {X}.

4. Recommencer jusqu'à ce qu'on n'ajoute rien de nouveau dans les ensembles PREMIER.

Calcul de Premier pour une chaîne

1. Pour calculer Premier pour une chaîne $X_1X_2\ldots X_n$:

i) Ajouter à Premier($X_1X_2\ldots X_n$) tous les symboles de Premier(X_1) différents de ε.

ii) Si ε est dans Premier(X1), ajouter également les symboles de Premier(X2) différents de ε, ...

iii) Finalement, si quelque soit i, Premier(X_i) contient ε, ajouter ε à Premier($X_1X_2\ldots X_n$).

3. Calcul de SUIVANT

Pour tout non-terminal A, SUIVANT(A) est l'ensemble de tous les symboles terminaux a qui peuvent apparaître immédiatement à droite de A dans une dérivation : $S \xrightarrow{*} \alpha A a \beta$ où α et β sont des chaînes de symboles grammaticaux.

Suivant(A) = $\{\mathbf{a} \in V_T / \exists\ \alpha, \beta \in V^*, S \xrightarrow{*} \alpha A \mathbf{a} \beta\}$

Exemple

G = ({a, b, c, d, ε}, {S, B, P}, S, R)

Où R : $\begin{cases} S \rightarrow Sc \mid Ba \\ B \rightarrow Pa \mid bPb \mid P \mid \varepsilon \\ P \rightarrow dS \end{cases}$

$S \rightarrow S\ c$ alors c ∈ SUIVANT(S)

$S \rightarrow Ba \rightarrow Pa \rightarrow dSa$ alors a ∈ SUIVANT(S)

$S \rightarrow Ba \rightarrow bPba \rightarrow bdSba$ alors b ∈ SUIVANT(S)

3. 1. Algorithme de construction des ensembles SUIVANT

Pour calculer SUIVANT(A) pour tous les non-terminaux, appliquer les règles suivantes jusqu'à aucun terminal ne puisse être ajouté aux ensembles SUIVANT :

1. Mettre $ dans SUIVANT(S) où S est l'axiome de départ de la grammaire et $ est le marqueur indiquant la fin du texte source.

2. S'il y a une production $A \rightarrow \alpha B \beta$ où B est un non-terminal, alors le contenu de PREMIER(β) sauf ε est ajouté à SUIVANT(B).

SUIVANT(B) ⊇ PREMIER(β) \ ε en effet : Si a ∈ Premier(β)\{ε} ($\beta \xrightarrow{*} aw$)

$S \xrightarrow{*} \alpha' A \beta' \rightarrow \alpha' \alpha B \beta \beta' \xrightarrow{*} \alpha' \alpha B a w \beta'$

3. S'il existe une production $A \rightarrow \alpha B$ ou $A \rightarrow \alpha B \beta$ telle que ε ∈ PREMIER(β) (ie $\beta \xrightarrow{*} \varepsilon$) alors les éléments de SUIVANT(A) sont ajoutés à SUIVANT(B).

SUIVANT(B) ⊇ SUIVANT(A) en effet : si a ∈ SUIVANT(A) :

$S \xrightarrow{*} \alpha' A a \beta' \rightarrow \alpha' \alpha B a \beta$ alors a ∈ SUIVANT(B).

$S \xrightarrow{*} \alpha' A a \beta' \rightarrow \alpha' \alpha B \beta a \beta' \xrightarrow{*} \alpha' \alpha B a \beta'$

car ε ∈ PREMIER(β) alors a ∈ SUIVANT(B).

4. Table d'analyse

Exemple 1

Soit la grammaire G définie par : S → aSb | aSc | d. Le mot analysé est w = *aaaaadbbcbbbc*.

Pour savoir quelle règle utilisée, il faut connaître la **dernière** lettre du mot.

Exemple 2

Reprenons la grammaire suivante : E → T E' E' → + T E' | ε T → F T' T' → * F T' | ε F → (E) | nb. Le mot analysé est w = 3*4+10*(5+11). La question posée est quelles sont les règles qui vont être utilisées?

4. 1. Construction d'analyseur

L'algorithme suivant peut être utilisé pour construire une table pour l'analyse prédictive d'une grammaire G. L'idée sous-tendant cet algorithme est le suivant.

• Soit A → α une production et a dans Premier(α).

• L'analyseur développe A en α chaque fois que le symbole d'entrée courant est a.

• Une production se produit quand α = ε ou α *→ ε. Dans ce cas, nous devons également développer A en α si le symbole d'entrée courant est dans SUIVANT(A) ou si le $ d'entrée a été atteint et que $ est dans SUIVANT(A).

4. 2. Algorithme

Donnée : une grammaire G.

Résultat : une table d'analyse M pour G.

Méthode :

• Pour chaque production A → α de la grammaire :

- Pour chaque terminal a dans Premier(α), ajouter A → α à M[A; a]

• Si ε est dans PREMIER (α) :

- Ajouter A → α à M[A; b] pour chaque terminal b dans Suivant(A)
- Si $ est dans Suivant(A), ajouter A → α à M[A; $].

• Faire de chaque entrée indéfinie de M une erreur de syntaxe.

Exemple

$E \to T\,E'$ $E' \to +\,T\,E'$ $E' \to \varepsilon$ $T \to F\,T'$ $T' \to *\,F\,T'$ $T' \to \varepsilon$ $F \to (E)$ $F \to nb$

Non-terminal	nb	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E′		$E' \to +TE'$			$E' \to \varepsilon$	$E' \to \varepsilon$
T	$T \to FT'$			$T \to FT'$		
T′		$T' \to \varepsilon$	$T' \to *FT'$		$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to nb$			$F \to (E)$		

5. Analyseur syntaxique dirigé par table

• Un **tampon d'entrée** : contient la chaîne à analyser, suivie de $, symbole utilisé comme marqueur d'extrémité droite qui indique la fin de la chaîne d'entrée.

• Une **pile** : contient une séquence de symboles grammaticaux, avec $ marquant le fond de pile. Initialement, la pile contient l'axiome de la grammaire au dessus du $.

• Une **table d'analyse** : tableau à deux dimensions M[A; a] où A est un non-terminal et a un terminal ou le symbole $.

• Un **flot de sortie** : ce qui est récupéré.

5. 1. Principe

L'analyse syntaxique est contrôlée par un programme qui a le comportement suivant, sachant que X est le symbole au sommet de la pile, et a le symbole d'entrée courant. Ces deux symboles déterminent l'action de l'analyseur. Il y a trois possibilités :

• Si X = a = $: l'analyseur s'arrête et annonce la réussite finale de l'analyse.

• Si X = a ≠ $: l'analyseur enlève X de la pile et avance son pointeur d'entrée sur le symbole suivant.

• Si X est un non-terminal, le programme consulte l'entrée M[X; a] de la table d'analyse M. Cette entrée sera soit une X-production de la grammaire soit une erreur.

> • Si par exemple, M[X; a] = {X → UVW}, l'analyseur remplace X en sommet de pile par WVU (avec U au sommet). Nous supposerons que l'analyseur se contente pour tout résultat d'imprimer la production utilisée.
>
> • Si M[X; a] = erreur, l'analyseur appelle une procédure de récupération sur l'erreur.

5. 2. Algorithme d'analyse

Donnée : une chaîne w et une table d'analyse M pour une grammaire G.

Résultat : si w est dans L(G), une dérivation gauche pour w, sinon une indication d'erreur.

Méthode : initialement, l'analyseur est dans une configuration dans laquelle il y a $S dans la pile, avec S l'axiome de G au sommet, et w$ dans son tampon d'entrée.

Algorithme

• Positionner le pointeur source ps sur le premier symbole de w$.

• Répéter jusqu'à ce que X = $.

- Soit X le symbole en sommet de pile et a le symbole pointé par ps.
- Si X est un terminal ou $ alors :
  - Si X = a alors enlever X de la pile et avancer ps.
  - Sinon Erreur ()
- Sinon (X est un non-terminal) :

  Si M[X; a] = $X \rightarrow Y_1Y_2 \dots Y_n$ alors

  Debut :

  Enlever X de la pile

  Mettre $Y_n; Y_{n-1}; \dots ; Y_1$ sur la pile (avec Y_1 au sommet)

  Emettre en sortie la production $X \rightarrow Y_1Y_2 \dots Y$

  Fin

  Sinon Erreur ()

Exemple

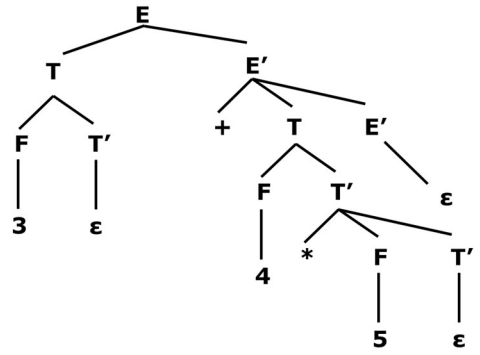
$E \rightarrow T\ E'$ $E' \rightarrow +\ T\ E'$ $E' \rightarrow \varepsilon$ $T \rightarrow F\ T'$ $T' \rightarrow *\ F\ T'$ $T' \rightarrow \varepsilon$ $F \rightarrow (E)$ $F \rightarrow nb$

Le mot w = 3 + 4 * 5

Pile	Entrée	Sortie
$ E	3 + 4 * 5 $	$E \rightarrow T\ E'$
$ E' T	3 + 4 * 5 $	$T \rightarrow F\ T'$

$ E' T' F	3 + 4 * 5 $	F → nb
$ E' T' **3**	**3** + 4 * 5 $	
$ E' T'	+ 4 * 5 $	T' → ε
$ E'	+ 4 * 5 $	E' → + T E'
$ E' T +	+ 4 * 5 $	
$ E' T	4 * 5 $	T → F T'
$ E' T' F	4 * 5 $	F → nb
$ E' T' **4**	**4** * 5 $	
$ E' T'	* 5 $	T' → * F T'
$ E' T' F *	* 5 $	
$ E' T' F	5 $	F → nb
$ E' T' **5**	**5** $	
$ E' T'	$	T' → ε
$ E'	$	E' → ε
$	**$**	Analyse réussie

Par conséquent, l'analyse syntaxique s'est bien déroulée er l'arbre de dérivation est construit et est définit ci-dessous :



Arbre syntaxique pour 3 + 4 *5.

Remarque

L'algorithme précédent peut il être appliqué à toutes les grammaires?

Si la table d'analyse comporte des entrées multiples (plusieurs productions pour une même case M[A, α]), le choix de la production adéquate sera difficile.

Définition

Une grammaire est dite de type LL(1) si la table d'analyse associée n'a aucune case définie de façon multiple.

Remarque

La grammaire n'est pas de type LL(1). En effet, pour pouvoir choisir entre la production A $\rightarrow$ cd et la production A $\rightarrow$ c, il faut lire la lettre qui suit celle que l'on pointe (donc deux symboles de prévision sont nécessaires).

Théorème

Une grammaire ambiguë ou récursive à gauche ou non factorisée à gauche n'est pas LL(1).

6. Récursivité à gauche

Une grammaire est immédiatement récursive à gauche si elle contient un non-terminal A tel qu'il existe une production A $\rightarrow$ Aα où α est une chaîne quelconque.

Exemple

S $\rightarrow$ ScA | B A $\rightarrow$ Aa | ε B $\rightarrow$ Bb | d | e

Cette grammaire contient plusieurs récursivités à gauche immédiates (ou directes).

6. 1. Elimination de la récursivité à gauche immédiate

Remplacer toute règle de la forme A $\rightarrow$ Aα | β avec $\alpha \in (V_T \cup V_N)^+$ et $\beta \in (V_T \cup V_N)^*$ par les deux règles :

$A \rightarrow \beta\ A'$

$A' \rightarrow \alpha\ A' \mid \varepsilon$

Algorithme

Dans un premier temps, on groupe les A-productions comme suit :

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$

où aucun β_i ne commence par un A. Puis on remplace les A-productions par :

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon$

Théorème

La grammaire ainsi obtenue reconnaît le même langage que la grammaire initiale.

7. Factorisation à gauche

Exemple

$S \rightarrow$ bacdAbd | bacdBcca

$A \rightarrow$ aD

$B \rightarrow$ cE

$C \rightarrow \ldots$

Au départ pour savoir s'il faut choisir $S \rightarrow$ bacdAbd ou $S \rightarrow$ bacdBcca, il faut lire la 5ième lettre du mot. Dès le départ, quelle production va être choisie? Ce qui est incompatible avec les grammaires LL(1).

Cas général

Si $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ sont deux A-productions et si l'entrée commence par une chaîne non vide dérivée de α, nous ne savons pas s'il faut développer A en $\alpha\beta_1$ ou A en $\alpha\beta_2$. Il est possible de différer cette décision en étendant A en $\alpha A'$. Alors après avoir lu la chaîne dérivée de α, nous étendons A' en β_1 ou en β_2.

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ devient $A \rightarrow \alpha A'$ $\quad A' \rightarrow \beta_1 \mid \beta_2$

7. 1. Algorithme

Pour chaque non-terminal A, trouver le plus long préfixe α commun à deux de ses alternatives ou plus.

Si $\alpha \neq \varepsilon$, remplacer $A \rightarrow \alpha\beta_1 \mid \ldots \mid \alpha\,\beta_p \mid \gamma_1 \mid \ldots \mid \gamma_p$

(où les γ_i ne commencent pas par α) par les deux règles :

$A \rightarrow \alpha A' \mid \gamma_1 \mid \ldots \mid \gamma_p$

$A' \rightarrow \beta_1 \mid \ldots \mid \beta_p$

fin si

Recommencer jusqu'à ne plus en trouver.

Bibliographie

Emmanuel Desmontils (IUP-MIAGE de Nantes)
Thomas Pietrzak, Université Paul Verlaine / Metz
…